JETBRAINS | Kotlin

Exceptions

kotlinglang.org/education



What? Why?

An exception signals that something went *exceptionally* wrong.

- Development mistakes
- Errors produced by external (to the program) resources
- System errors

Why use exceptions:

- To separate error-handling code from regular code
- To propagate errors up the call stack – maybe someone knows how to deal with the error
- To group and differentiate error types

Do NOT use exceptions for:

- Control flow
- Manageable errors

How?

```
fun main() {
   throw Exception("Hello, world!")
}
```

Or even better:

```
fun main() {
   val nullableString: String? = null
   println("Hello, NPE! ${nullableString!!}")
}
Exception in thread "main"
java.lang.NullPointerException
```

Example

```
fun main() {
     try {
        throw Exception("An exception",
RuntimeException("A cause"))
     } catch (e: Exception) {
        println("Message: ${e.message}")
        println("Cause: ${e.cause}")
        println("Exception: $e") // toString() is
called "under the hood"
        e.printStackTrace()
     } finally {
        println("Finally always
executes")
     }
}
```

```
Message: An exception
Cause:
java.lang.RuntimeException: A cause

Exception: java.lang.Exception: An exception
Finally always executes

java.lang.Exception: An exception
at MainKt.main(Main.kt:3)

at MainKt.main(Main.kt)

Caused by: java.lang.RuntimeException: A cause

2 more
```

Another meaningful example

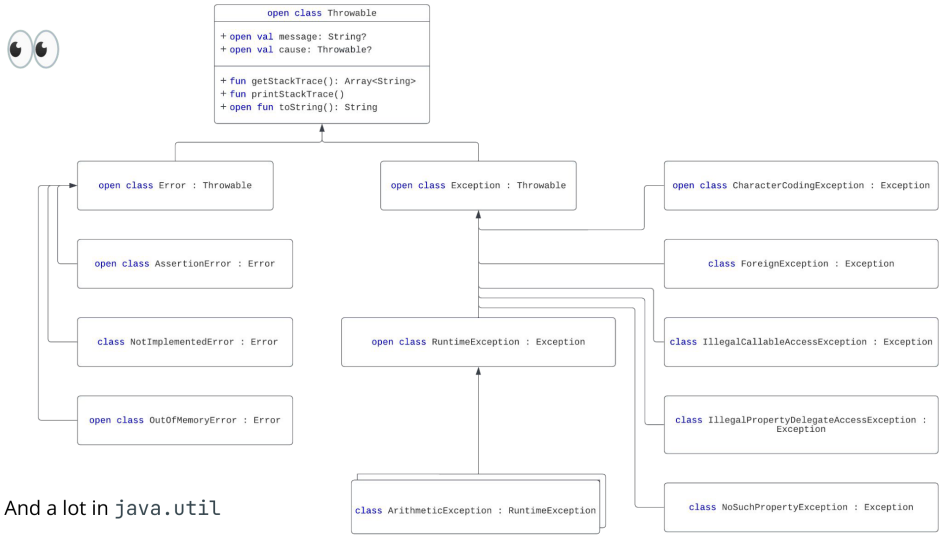
```
data class Person(val name: String, val
surname: String, val age: Int) {
   init {
       if (age < 0) {
           throw
IllegalStateException("Age cannot be
negative")
       }
       if (name.isEmpty() ||
surname.isEmpty()) {
           throw
IllegalArgumentException("For blank names/
surnames use -")
       }
   }
}
```

Dealing with exceptions

```
try {
   val (n, s, a) = readLine()!!.split('/')
   val person = Person(n, s, a.toInt())
   addToDataBase(person)
} catch (e: IllegalStateException) {
   println("You've entered a negative age! Why?")
} catch (e: IllegalArgumentException) {
   println(e.message)
} catch (e: NullPointerException) {
   println("NPE ;^)")
} catch (e: Exception) {
   println("Something else went wrong")
   throw Exception("Failed to add to the database", e)
} finally {
   println("See you in the next episodes!")
}
```

You might:

- Handle the error properly and continue execution
- Handle something on your side and re-throw the exception



And a lot in `java.util`

Kotlin sugar

`try` is an expression:

```
val a: Int? = try { input.toInt() } catch
(e: NumberFormatException) { null }
```

More sugar:

```
require(count ⩾ 0) { "Count must be non-
negative, was $count" }
// IllegalArgumentException

error("Error message")
// IllegalStateException
```

JETBRAINS | Kotlin

Thank you

